ARES: A Framework for Measuring Reliability and Failure Modes in Autonomous AI Security Agents

Mohammad Makchudul Alam[1,2], Md. Redowan Zaman Anik[2], Sabrein Serag El Din el Bodour[2], Moshiul Islam[3], and Anca Delia Jurcut[4]

[1] *DNS Research Lab, University College Dublin* [2] *BGD e-GOV CIRT, Dhaka, Bangladesh*
[3] *EIC Limited, Dhaka, Bangladesh* [4] *School of Computer Science, University College Dublin, Ireland*

Abstract—AI agents triage alerts in Security Operations Centres (SOCs), yet their failure conditions are poorly understood. ARES models agent failures as skill, knowledge, and capability mismatches. Using ARES-BENCH and SOCAGENTFAILURE-1K, we show knowledge mismatch drives hallucination, mismatch dimensions produce distinct failure signatures, and multi-agent pipelines amplify reliability risk.

Index Terms—AI agent reliability, cybersecurity, failure taxonomy, hallucination, SOC automation, epistemic gap

1 INTRODUCTION

Security Operations Centres (SOCs) are undergoing a fundamental transformation. Autonomous AI agents (powered by large language models (LLMs) and equipped with tool-use capabilities) now perform threat detection, alert triage, and incident response tasks that were previously the exclusive domain of human analysts [1]. Frameworks such as LangGraph [2] and AutoGen [3] have made it straightforward to deploy multi-step reasoning agents that query SIEM systems, correlate indicators of compromise (IOCs), and recommend containment actions with minimal human oversight. Commercial platforms including CrowdStrike Charlotte AI and SentinelOne Purple AI have begun embedding agent-based automation into production SOC workflows, and the trend is accelerating.

Yet a critical question remains unanswered: *when do these agents fail, and how dangerous are those failures?* The security community has focused heavily on measuring what AI agents can do (detection accuracy, response speed, ATT&CK coverage) while paying far less attention to the conditions under which agents produce incorrect, incomplete, or actively misleading outputs. This gap carries serious operational consequences:

- A **false negative** allows an intrusion to persist undetected for hours or days, expanding the attacker's foothold while the SOC believes the environment is clean.
- A **hallucinated threat attribution** propagates incorrect IOCs into shared intelligence feeds, poisoning downstream defences across multiple organisations that consume the same threat data.
- A **delayed response**, caused by an agent paralysed by missing tool access or overwhelmed by input complexity, allows an incident to escalate beyond regulatory reporting thresholds.

These are not hypothetical scenarios: they are predictable consequences of deploying agents whose capabilities do not match the demands of their assigned tasks. We call this the *Skill–Knowledge–Capability (SKC) mismatch* problem: the systematic gap between what a security task requires and what an agent can actually deliver across multiple dimensions including procedural skills, declarative threat knowledge, tool access, reasoning quality, and knowledge currency.

Existing work addresses fragments of this problem in isolation. Classical dependability theory [4] predates LLM agents and has no notion of epistemic uncertainty or knowledge currency. Hallucination research [5] treats reasoning failures as model-internal properties without modelling tool availability, task complexity, or knowledge freshness. Calibration work [6] shows neural networks are miscalibrated but does not connect this to agent reliability. Agent benchmarks [7], [8] measure capability without modelling failure conditions. The closest recent efforts — MAST [9], a domain-agnostic 14-mode failure taxonomy, and AgentDojo [10], an adversarial prompt-injection environment — each cover one slice of the surface. To the best of our knowledge, no existing work provides a unified reliability model, evaluation testbed, and labelled failure dataset for the full non-adversarial failure surface of AI security agents.

This paper presents ARES (Agentic Reliability Evaluation for Security systems), a framework designed to fill this gap. ARES provides:

1) A **three-axis failure taxonomy** classifying AI security agent failures by origin (8 classes), manifestation (8 classes), and operational impact (7 classes), grounded in CSIRT practice.
2) A **quantitative reliability model** based on SKC mismatch, including an Agent Reliability Score (ARS) with task-class-specific weights and an Epistemic Gap metric that formalises the hallucination precondition.
3) **ARES-BENCH**, a fully open-source, Docker-deployable evaluation testbed supporting four agent architectures across five threat scenario classes.

4) **SOCAGENTFAILURE-1K**, a labelled dataset of 1,000 agent execution traces under controlled failure conditions, released under CC BY 4.0.

Together they let SOC architects screen an agent for a given task before it ever sees production traffic.

Parallel 2025–2026 efforts confirm the urgency: hallucination benchmarks [11] treat confident factual errors as a cross-domain failure class, and the OWASP Top 10 for Agentic Applications [12] elevates capability-mismatch and tool-misuse failures to first-class operational risks. None of these efforts connect agent reliability to SOC-specific consequences: a misclassified ransomware beacon or a fabricated CVE attribution. ARES fills that gap.

2 Why Reliability of AI Agents Matters for Cybersecurity

The reliability of AI agents is not merely an academic concern: it is an operational safety requirement. When an AI agent operates autonomously in a SOC, its outputs directly influence incident response decisions that affect the security posture of entire organisations. Unlike traditional software, where bugs produce deterministic incorrect outputs, LLM-based agents introduce a qualitatively different failure surface: *stochastic, context-dependent, and confidence-inflated errors* that are difficult to detect and dangerous to act upon.

Risks of autonomous AI decisions. A human analyst who encounters an unfamiliar threat pattern will typically escalate or seek additional context. An LLM-based agent, by contrast, may confidently classify a novel threat using superficially similar patterns from its training data, a failure mode known as hallucination. In cybersecurity, hallucination is particularly dangerous because the domain is dense with specific factual claims (CVE identifiers, threat actor names, TTP chains) that are easy to fabricate convincingly but difficult to verify in real-time [5]. The consequences of incorrect autonomous decisions are often irreversible: once a compromised host is left un-isolated or a fabricated IOC enters a shared feed, the damage propagates faster than human review can contain it.

Impact on incident response. Consider a three-stage agent pipeline: triage, enrichment, and remediation recommendation. If the triage agent misclassifies a ransomware intrusion as a benign false positive, the enrichment agent never investigates, and the remediation agent recommends no action. The intrusion persists, the attacker escalates privileges, and exfiltration proceeds undetected. Conversely, if the triage agent hallucinates a threat actor attribution (for example, incorrectly attributing a commodity phishing campaign to an APT group) the enrichment agent may propagate fabricated IOCs to shared intelligence feeds such as STIX/TAXII repositories, corrupting defences across every organisation that consumes those feeds. In a third scenario, an agent that lacks access to the EDR containment API (capability mismatch) may correctly identify the threat but fail to execute the recommended isolation action, introducing a critical delay in response.

Trust and accountability. SOC analysts must trust AI agent outputs enough to act on them, but not so much that they stop verifying. When agents fail silently, producing plausible but incorrect outputs, they erode this trust calibration over time. Repeated false confidence leads analysts to either over-rely on AI-assisted workflows (automation complacency) or disengage from them entirely (automation aversion), both of which degrade operational effectiveness. The challenge is compounded in organisations subject to regulatory frameworks such as NIS2 or DORA, where accountability for incident response decisions must be traceable regardless of whether a human or an AI agent made the initial assessment.

Why hallucination is uniquely dangerous in SOC environments. General-purpose hallucination is problematic; hallucination in security operations is *actionable and consequential*. A hallucinated medical diagnosis triggers further tests before treatment. A hallucinated legal citation is caught during review. But a hallucinated threat attribution may trigger immediate containment actions (host isolation, network segmentation, threat feed updates) with real operational cost and often no natural verification checkpoint before execution. The combination of high confidence, domain-specific factual density, and automated downstream actions makes SOC hallucination uniquely dangerous among AI application domains.

Why existing assurance approaches fall short. Classical software testing does not catch stochastic, context-dependent agent failures: a test suite that passes at build time says nothing about reliability under a new CVE or novel ransomware variant. SOC peer review is the safety net for sensitive decisions, but autonomous agents are deployed precisely to remove that bottleneck. Adversarial ML benchmarks measure robustness against crafted perturbations rather than against the benign-but-unfamiliar inputs that dominate SOC workload. Hallucination benchmarks [11] count how often models fabricate facts but do not connect fabrication to the downstream security actions that turn a wrong answer into operational harm. These gaps motivate a structured framework for measuring agent reliability *before deployment* and monitoring it *during operation* — which is what ARES provides.

3 Failure Taxonomy for AI Security Agents

Understanding *how* agents fail is a prerequisite for measuring reliability. We propose a three-axis taxonomy that decomposes each failure event along three orthogonal dimensions: what caused it (*origin*), what it looked like (*manifestation*), and what it cost (*impact*). Unlike prior fault taxonomies developed for conventional software [4], this taxonomy is designed specifically for autonomous AI security agents, where failures are not bugs to be patched but operational risks that affect real-world security outcomes. Table 1 summarises the three axes.

Two distinctions matter for practitioners. First, *Hallucination* (M1) and *Confabulation* (M4) are not synonyms. Hallucination asserts atomic facts that are verifiably false against a closed-world corpus; confabulation produces internally coherent reasoning founded on non-existent premises. Confabulation is more dangerous because it is harder to detect by automated checks and more persuasive to analysts reviewing the agent's reasoning trace. Second, *Intelligence Pollution* (I5) is qualitatively unlike the other impacts: a single

TABLE 1
Three-Axis Failure Taxonomy for AI Security Agents

Code	Class	Description
Axis 1: Origin (root cause)		
O1	Epistemic Deficiency	Lacks required threat knowledge
O2	Skill Boundary	Task requires an unlearned procedure
O3	Capability Deprivation	Required tool or API unavailable
O4	Reasoning Corruption	Inference produces invalid output
O5	Context Overload	Input exceeds processing capacity
O6	Adversarial Perturbation	Malicious content degrades output
O7	Orchestration Failure	Multi-agent coordination breakdown
O8	Temporal Drift	Knowledge has become stale
Axis 2: Manifestation (observable signal)		
M1	Hallucination	Asserts verifiably false facts
M2	Omission	Fails to detect a present threat
M3	Misclassification	Wrong threat family or category
M4	Confabulation	Coherent reasoning from false premises
M5	Paralysis	No output or persistent retry loop
M6	Overclaiming	High confidence on weak evidence
M7	Attribution Error	Wrong threat actor or campaign
M8	Cascade Failure	Error propagates across agents
Axis 3: Impact (operational consequence)		
I1	False Negative *	Threat undetected; attacker persists
I2	False Positive	Benign activity flagged
I3	Delayed Response	Action issued past SLA
I4	Wrong Remediation *	Incorrect containment executed
I5	Intelligence Pollution *	Fabricated IOCs propagate to feeds
I6	Trust Erosion	Analyst confidence reduced
I7	Compliance Violation	Regulatory threshold breached

* Critical impact category.

hallucinated IOC pushed to a STIX/TAXII feed propagates beyond the originating organisation to the entire information-sharing community.

The power of the three-axis decomposition is its diagnostic composability. A practitioner who observes a cluster of M1 events can trace back to O1 (knowledge deficiency) or O5 (context overload) as the most probable origins, and forward to I1 (false negative) or I5 (intelligence pollution) as the likely consequences. Each origin maps to a distinct operational action: O1 calls for knowledge refresh, O5 for input preprocessing, O3 for tool provisioning.

Worked example: Intelligence Pollution. A multi-agent pipeline correctly identifies a phishing campaign, but the enrichment stage invents three IOC URLs by extrapolating from a domain pattern it has seen before. A downstream automation pushes these IOCs to the shared threat feed; within minutes, partner SOCs block legitimate subdomains of a genuine customer-service platform. The trace decomposes cleanly: origin is O1 (the enrichment agent lacked the TTP chain required to enumerate IOCs correctly); manifestation is M1 (hallucinated factual claims); impact is I5, with a blast radius far beyond the originating organisation. The same trace motivates three design decisions simultaneously: per-IOC confidence gating, confidence-proportional feed propagation, and human-in-the-loop review for feed writes — each traceable to a distinct axis of the taxonomy.

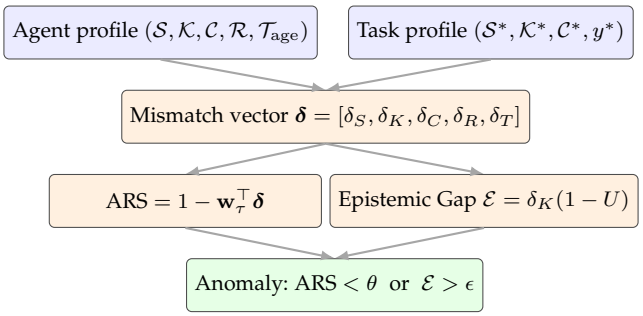

Fig. 1. The ARES reliability pipeline. The agent's declared profile and the task's requirement profile produce a five-dimensional mismatch vector that feeds two parallel indicators: the Agent Reliability Score and the Epistemic Gap. Either crossing its learned threshold flags the execution as anomalous.

4 Reliability Model: SKC Mismatch

The taxonomy tells us *how* agents fail; the reliability model tells us *why* and *how likely*. At its core, ARES measures the gap between what a security task demands and what an agent can deliver across five dimensions. The model extends classical dependability theory [4] by introducing knowledge incompleteness, probabilistic reasoning errors, and capability constraints as first-class failure sources: dimensions that do not exist in traditional deterministic software systems. Figure 1 summarises the end-to-end flow.

4.1 Five Mismatch Dimensions

For each dimension, we compute a normalised mismatch score between 0 (full coverage) and 1 (total mismatch):

- **Skill mismatch** (δ_S): the fraction of required procedural skills the agent lacks. For example, a log-correlation agent assigned binary malware analysis has a high δ_S because the required analytical procedure is outside its repertoire.
- **Knowledge mismatch** (δ_K): the fraction of required threat knowledge missing from the agent's context, measured over the MITRE ATT&CK [13] knowledge graph. An agent unaware of a recently emerged ransomware variant has high δ_K for tasks involving that family.
- **Capability mismatch** (δ_C): the fraction of required tools or APIs the agent cannot access. An agent that recommends host isolation but lacks EDR API access has high δ_C: it knows *what* to do but cannot *do* it.
- **Reasoning mismatch** (δ_R): the divergence between the agent's output and ground truth, capturing inference quality. This dimension is unique in that it can only be measured post-execution.
- **Temporal drift** (δ_T): knowledge staleness modelled as exponential decay, parameterised by the threat domain's volatility. Ransomware intelligence ($\lambda = 0.005$ day^{-1}) decays rapidly; APT attribution knowledge ($\lambda = 0.001$ day^{-1}) remains stable for longer.

A critical design property: the first three dimensions (δ_S, δ_K, δ_C) and temporal drift (δ_T) can all be computed *before* task execution from the agent's declared profile and the task's requirements. In deployment, ARES therefore uses skill, knowledge, capability, and temporal drift for pre-execution screening; reasoning mismatch (δ_R) is used after execution for diagnosis and model improvement.

4.2 Agent Reliability Score

The five mismatch scores are combined into a single **Agent Reliability Score (ARS)**:

$$\mathrm{ARS} = 1 - \mathbf{w}_\tau^\top \boldsymbol{\delta}$$

where $\boldsymbol{\delta} = [\delta_S, \delta_K, \delta_C, \delta_R, \delta_T]$ is the mismatch vector and $\mathbf{w}_\tau$ is a weight vector specific to the threat class τ, learned via ridge regression on labelled training data. An ARS of 1.0 indicates perfect alignment between agent capability and task demand; values below a learned threshold θ^* flag the agent as unreliable for that task.

The weights are learned per threat class because different security tasks stress different dimensions. In our experiments, knowledge coverage ($w_K = 0.35$) dominates for ransomware detection, while skill and capability access ($w_S = 0.32$, $w_C = 0.28$) lead for DDoS mitigation, and knowledge combined with reasoning ($w_K = 0.38$, $w_R = 0.24$) lead for APT intrusion investigation. A single universal weight vector would systematically misestimate reliability by at least 12% on certain threat classes.

4.3 Epistemic Gap: The "Confident Ignorance" Problem

The most dangerous agent failures occur not when an agent lacks knowledge, but when it *does not know that it lacks knowledge*, a failure of confidence calibration [6]. We formalise this as the **Epistemic Gap**:

$$\mathcal{E} = \delta_K \times (1 - U)$$

where U is the agent's expressed uncertainty (complement of its stated confidence). An agent with a large knowledge deficit ($\delta_K \approx 1$) that simultaneously reports high confidence ($U \approx 0$) produces a maximal Epistemic Gap: the precondition for hallucination. Conversely, an agent that correctly identifies its knowledge limits ($U \approx 1$) produces a near-zero Epistemic Gap: it may fail the task but will not fabricate an answer.

This concept relates directly to the confidence calibration literature, which has shown that modern neural networks (including LLMs) are systematically overconfident on out-of-distribution inputs [6]. The Epistemic Gap operationalises this finding for the specific case of agentic AI: it measures not just whether the model is miscalibrated in general, but whether that miscalibration occurs precisely where the agent's knowledge is weakest: the most dangerous combination.

The distinction is critical for SOC operations. An agent that says "I don't have sufficient context to classify this threat" is safe: it triggers human escalation. An agent that confidently asserts a false threat attribution is dangerous: it triggers automated actions based on fabricated intelligence.

4.4 Cascade Reliability in Multi-Agent Systems

When agents are composed into pipelines, failures propagate through information dependencies. We model this using a directed acyclic graph where each edge represents an inter-agent data flow, and show that cascade reliability is bounded above by the *weakest agent* in the pipeline.

This result has a counterintuitive practical consequence: decomposing a security task into a multi-agent pipeline cannot improve reliability beyond the weakest component agent. It can only reduce it. A triage agent that hallucinates an IOC passes corrupted input to the enrichment agent, which inherits the error, potentially amplifies it by adding supporting (but irrelevant) context, and the recommendation agent acts on the compounded misinformation. Our empirical results decompose this effect cleanly. The naive head-line gap between single-agent LangGraph and three-stage AutoGen is $\sim 5.7\%$ relative ARS, but it conflates framework identity with pipeline depth. A controlled ablation on 27 strictly matched mismatch buckets isolates the two contributions: framework identity contributes $\mathbf{0.13}\%$ (essentially zero) and pure pipeline depth contributes $\mathbf{17.25}\%$. The cascade-induced reliability loss is therefore an order of magnitude larger than the head-line number suggests, and it is attributable entirely to multi-stage composition rather than to the choice of agent framework. Full ablation numbers are reported in the empirical-insights section.

4.5 From Score to Action

Operationally, an ARS assessment runs at task-assignment time and yields three control points. If ARS clears the threshold, the agent runs autonomously. If it falls short, the decomposition itself identifies the cheapest mitigation: a high δ_K calls for a knowledge refresh, a high δ_C for tool provisioning, and a high δ_T for a scheduled retraining cadence. The Epistemic Gap is computed at output time and gates downstream propagation: high-$\mathcal{E}$ outputs go to human review rather than to automated containment or shared threat feeds. Because four of the five dimensions are observable before execution, the screening cost is effectively zero.

5 ARES-BENCH EVALUATION FRAMEWORK

To validate the reliability model empirically, we built ARES-BENCH: an open-source, Docker-deployable testbed for measuring AI security agent reliability under controlled conditions. The design prioritises reproducibility (single-command deployment, all data sources public and versioned), generalisability (multiple agent architectures and threat classes), and controllability (systematic mismatch injection across a structured matrix). The full architecture diagram, data-source manifest, and measurement-pipeline details are maintained in the project artefact repository.[1]

Agents under test. ARES-BENCH evaluates four agent architectures spanning the major paradigms: **AUT-1 (LangGraph)** [2], a stateful graph-based ReAct loop; **AUT-2 (AutoGen)** [3], a three-stage multi-agent pipeline (triage $\rightarrow$ enrichment $\rightarrow$ recommendation); **AUT-3 (CrewAI)**, a role-based crew (analyst, enrichment, incident commander); and

1. https://github.com/IntelEyes/ARES-Framework

AUT-4, a deterministic Sigma/YARA evaluator with no LLM component, serving as the $\delta_R = 0$ control. Hallucination is structurally impossible in AUT-4. All LLM-based agents use locally deployed Mistral-7B-Instruct via Ollama.

Controlled mismatch injection. A Mismatch Injector applies controlled degradation across four dimensions ($\delta_S, \delta_K, \delta_C, \delta_R$) at four levels each (0, 0.33, 0.66, 1.0). Mechanisms are deliberately realistic: disabling tool nodes in the agent's execution graph (δ_S); filtering ATT&CK TTP context from retrieval to retain only a controlled fraction (δ_K); intercepting API calls via a middleware proxy that returns permission-denied errors (δ_C); and inserting misleading content into the system prompt at controlled probability (δ_R). Temporal drift (δ_T) is set parametrically via the agent's knowledge-age attribute.

Failure measurement. An Anomaly Monitor parses agent output, cross-references claims against ATT&CK and NVD as a closed-world ground truth, and flags any unverifiable claim asserted with confidence > 0.70 as a hallucination candidate. It then computes the five mismatch dimensions, the ARS, and the Epistemic Gap, and assigns taxonomy labels using a rule-based engine validated against human annotations (Cohen's $\kappa = 0.84$ for origin, $\kappa = 0.91$ for manifestation).

The SOCAGENTFAILURE-1K dataset. The pipeline produces a labelled corpus of 1,000 agent execution traces stratified across five threat classes (200 each), four agent architectures (250 each), and 16 representative mismatch conditions. Each record captures the task context (scenario class, TTP chain, ground truth), the agent profile, the five-dimensional mismatch vector, the full agent output with reasoning trace, the failure-taxonomy labels, the ARS and Epistemic Gap scores, and execution metadata. The dataset is released under CC BY 4.0 alongside the ARES-BENCH codebase.

6 EMPIRICAL INSIGHTS

We summarise five findings from the SOCAGENTFAILURE-1K experiments most relevant to practitioners deploying AI agents in security environments. Each is paired with the operational implication it carries for SOC teams.

6.1 Knowledge Mismatch Drives Hallucination

Finding. Across 750 LLM-based agent executions, hallucination rate increases monotonically with knowledge mismatch (δ_K): from 1.4% at $\delta_K = 0$ to 39.1% at $\delta_K = 1.0$ (Pearson $\rho = 0.92$, $p < 0.001$). The pattern holds across all three LLM architectures (LangGraph, AutoGen, CrewAI). The rule-based baseline (AUT-4) hallucinates zero times regardless of δ_K, confirming hallucination as an LLM-specific failure mode.

Implication. Verify threat-knowledge coverage before autonomous execution. Knowledge gaps are the single strongest predictor of dangerous output fabrication, which makes retrieval-augmented generation (RAG) pipeline quality a first-order reliability concern: the completeness of the retrieved context directly determines hallucination risk.

TABLE 2
ARS Anomaly Detection Performance (Test Set)

Threat Class	Precision	Recall	F1
Ransomware	1.00	0.94	0.97
Phishing	1.00	0.94	0.97
Insider Threat	1.00	0.87	0.93
DDoS	1.00	0.84	0.91
APT Intrusion	1.00	1.00	1.00
Overall	**1.00**	**0.92**	**0.96**

6.2 Different Mismatches Produce Distinct Failure Signatures

Finding. Each mismatch dimension produces a diagnostically separable failure pattern (chi-square test of independence, $p < 0.001$). Conditioning on the LLM-agent records whose *primary* mismatch dimension is at level ≥ 0.5: capability deprivation (δ_C) predominantly produces *Paralysis* (78%); skill deficit (δ_S) produces silent *Omission* (72%); knowledge gaps (δ_K) produce *Hallucination* (32%) and *Misclassification* (21%); reasoning perturbation (δ_R) produces *Confabulation* (38%) and *Overclaiming* (17%).

Implication. The manifestation pattern is a diagnostic fingerprint. A SOC observing high paralysis should investigate tool access and API permissions, not LLM quality. A high confabulation rate suggests reasoning corruption or adversarial input manipulation rather than knowledge gaps. This enables targeted remediation rather than wholesale agent replacement.

6.3 ARS Generalises as a Reliability Gate

Finding. The learned ARS threshold ($\theta^* = 0.58$) achieves test F1 = 0.96 across all five threat classes (Table 2), with perfect precision (1.00) and recall ranging from 0.84 to 1.00. The minimal train-test gap (0.95 vs. 0.96) indicates that ARS captures a generalisable reliability signal rather than overfitting to specific threat classes or agent architectures. The reported F1 uses the post-execution ARS that includes δ_R; in deployment, the pre-execution gate uses δ_S, δ_K, δ_C, and δ_T, with δ_R added after execution for diagnosis and continuous improvement.

Implication. ARS works operationally as a pre-execution deployment gate. Perfect precision means no false alarms: every flagged execution truly exhibited anomalous behaviour, removing the common adoption blocker where reliability tooling generates so many false alerts that SOC teams disable it.

6.4 Multi-Agent Pipelines Amplify Failures

Finding. Comparing AUT-1 (single-agent LangGraph) with AUT-2 (three-stage AutoGen pipeline) shows a 5.7% headline ARS gap that conflates framework identity with pipeline depth. A controlled ablation on 27 strictly matched mismatch buckets isolates the two contributions: framework identity contributes **0.13%** (essentially zero) and pure pipeline depth contributes **17.25%** relative ARS reduction, consistent across all five threat classes.

Implication. Decomposing a security task into a multi-agent pipeline does not improve reliability; it reduces it whenever component agents carry mismatch vulnerabilities. Prefer

well-validated single-agent designs for critical tasks. If multi-agent is required, add inter-stage verification checkpoints and confidence-gated propagation so that low-confidence intermediate outputs do not silently seed downstream stages.

6.5 Epistemic Gap Separates Knowledge Risk from Confidence Risk

Finding. The Epistemic Gap ($\mathcal{E}$) and raw knowledge mismatch (δ_K) achieve statistically equivalent correlation with false-negative severity ($\rho_{\mathcal{E}} = \rho_{\delta_K} = 0.80$, Fisher z-test $p = 0.78$). On 30 live CISA KEV incidents beyond every model's training cutoff, five frontier LLMs scored identically on surface accuracy (vendor 67%, product 63%, CVE $\leq$3%), yet stated confidence diverged sharply: GPT-5.4-mini reported mean confidence 0.78 on 29/30 records while Claude Opus 4.7 abstained on 27/30.[2] Same accuracy, different operational risk.

Implication. The Epistemic Gap's value is not raw predictive lift but *interpretability*: it factorises risk into a "what the agent does not know" term (δ_K) and a "how confidently it acts under that ignorance" term ($1-U$), exposing two independently controllable mitigations. Calibration is therefore as actionable a lever as knowledge coverage. The hallucination-versus-δ_K relationship replicates directionally across four LLM families from 7B to frontier scale (bucket-aggregated $\rho \in [0.78, 0.95]$), so the reliability signal is structural rather than backbone-specific.

7 Deployment Guidance for SOC Teams

This is the practitioner section. ARES translates into concrete, actionable guidance for organisations deploying autonomous AI agents in security operations. We organise the guidance around five questions SOC architects and security engineers face when integrating agentic AI into their workflows, and close with a five-step pre-deployment checklist.

When should AI agents be trusted? ARS serves as a pre-execution deployment gate. An agent whose ARS falls below the learned threshold ($\theta^* = 0.58$) should not execute autonomously: escalate to a human analyst or reconfigure (provide additional knowledge context or restore tool access). Four of the five mismatch dimensions ($\delta_S, \delta_K, \delta_C, \delta_T$) are computable before execution, so screening occurs at task-assignment time with no runtime overhead. ARS checks integrate into SOAR platforms as a pre-condition for autonomous execution, consistent with emerging AI risk management frameworks [14].

How can reliability be monitored continuously? The Epistemic Gap enables runtime hallucination risk monitoring: by comparing the agent's stated confidence against its known knowledge coverage, a monitor flags high-risk outputs for human review before they trigger automated actions, valuable for host isolation, threat feed updates, and evidence preservation. Computing $\mathcal{E}$ requires only the agent's confidence and pre-computed δ_K, both already in the execution record.

How should knowledge maintenance be scheduled? The temporal drift model gives per-domain update schedules. For ransomware ($\lambda = 0.005$ day^{-1}), ARS drops below threshold in ~ 120 days (quarterly refresh). For APT attribution ($\lambda = 0.001$), the same threshold is not crossed for ~ 500 days. These directly inform agent-deployment runbooks.

How should multi-agent systems be designed? Cascade analysis shows pipelines amplify rather than mitigate component failures. Prefer single-agent designs for critical tasks, or independently verify per-agent reliability before composition. If multi-agent is required, three mitigations reduce cascade risk: (1) inter-agent verification checkpoints; (2) confidence-gated propagation (low-confidence outputs go to humans, not downstream); (3) redundant agent paths for critical decisions.

How can safer agent systems be built? Three principles emerge: (1) *knowledge-first design*, ensure comprehensive, current threat knowledge before optimising reasoning or tool access, since δ_K is the strongest single predictor of dangerous failures; (2) *calibrated uncertainty*, train or prompt agents to express uncertainty proportional to knowledge coverage, reducing the Epistemic Gap; (3) *fail-safe defaults*, configure agents to escalate rather than fabricate when operating outside their validated envelope. These apply broadly but are especially critical in cybersecurity where confident errors cascade beyond the immediate incident.

How does this map to regulatory and governance frameworks? The SKC decomposition aligns directly with emerging AI governance standards. The NIST AI Risk Management Framework [14] identifies "valid and reliable," "accountable and transparent," and "safe" as top-tier trustworthiness characteristics: ARES operationalises the first via ARS, the second via the traceable mismatch decomposition, and the third via the Epistemic-Gap escalation gate. The forthcoming NIST Cybersecurity and AI Profile [15] extends these controls explicitly to security-domain deployments, making pre-execution reliability gates an auditable practice rather than an internal engineering hygiene step. At the application layer, the OWASP Top 10 for Agentic Applications [12] catalogues capability over-provisioning, tool misuse, and confident hallucination as distinct risk categories, each of which has a direct analogue in ARES: δ_C over-provisioning, δ_C plus δ_R tool misuse, and the Epistemic Gap for confident hallucination.

A deployment checklist. Distilling the above into a pragmatic checklist for SOC architects yields five steps: *(i)* estimate pre-execution δ_S, δ_K, δ_C, δ_T for each intended task class; *(ii)* apply the task-class-specific weight vector and compare ARS against $\theta^* = 0.58$; *(iii)* if ARS is below threshold, choose one of three mitigations (knowledge refresh, capability provisioning, or task-scope restriction) and re-score; *(iv)* instrument the runtime to emit stated confidence alongside each decision and compute $\mathcal{E}$ per record; *(v)* schedule knowledge refreshes according to the domain-specific temporal-drift constant ($\sim$120 days for ransomware, $\sim$500 days for APT attribution in our data). Each step maps to a concrete engineering artefact (a config change, a telemetry hook, a cron job) so the reliability gate does not stay theoretical.

8 Limitations and Future Work

Synthetic benchmark data. SOCAgentFailure-1K is generated from ATT&CK and NVD using template-based

2. Model identifiers reflect the public API versions current at evaluation time (April 2026); `gpt-5.4-mini` and `claude-opus-4-7` are accessed through the OpenAI and Anthropic APIs respectively.

scenarios. Real SOC incidents introduce noise and edge cases this construction does not capture.

Closed-world hallucination labels. Hallucination labels treat ATT&CK and NVD as ground truth, so novel post-corpus facts may be incorrectly flagged as hallucinations.

Model-specific thresholds. Absolute ARS values and the $\theta^* = 0.58$ threshold were learned on Mistral-7B-Instruct and will require recalibration for larger backbones; directional findings should hold because the SKC model is architecture-agnostic.

Reasoning mismatch is diagnostic. The reasoning mismatch dimension (δ_R) is computed post-hoc from output comparison with ground truth; in production it must be estimated via proxy measures, making it a diagnostic rather than a pre-execution predictor. The remaining four dimensions remain fully prospective.

Future work extends ARES to larger LLMs, real-world SOC data, cyclic multi-agent architectures, and adversarial mismatch injection.

9 Conclusion

As AI agents take on greater operational responsibility in defending critical infrastructure, frameworks for understanding their failure behaviour become essential, not as an afterthought, but as a prerequisite for responsible deployment. ARES provides an integrated treatment of this problem for the cybersecurity domain: a failure taxonomy that classifies how agents fail across origin, manifestation, and impact dimensions; a quantitative reliability model based on skill–knowledge–capability mismatch; an open evaluation testbed supporting controlled experimentation; and a labelled failure dataset enabling reproducible benchmarking.

Our empirical results establish three findings with direct implications for SOC operations. First, knowledge gaps are the dominant driver of hallucination: agents operating without relevant threat context fabricate information at alarming rates, making knowledge completeness verification the highest-priority deployment check. Second, multi-agent pipelines amplify rather than mitigate component failures, challenging the intuition that task decomposition improves outcomes. Third, the Epistemic Gap (capturing what an agent does not know it does not know) decomposes false-negative risk into a knowledge term and a calibration term that are statistically equivalent in raw predictive power but independently controllable, making confidence calibration as actionable a lever as knowledge coverage.

These findings translate into actionable deployment guidance: pre-execution reliability screening via ARS, runtime confidence monitoring via the Epistemic Gap, evidence-based knowledge refresh scheduling via the temporal drift model, and conservative multi-agent pipeline design informed by cascade reliability analysis.

ARES, ARES-BENCH, and SOCAGENTFAILURE-1K are publicly released to support reproducible research in AI security agent dependability.[3]

3. https://github.com/IntelEyes/ARES-Framework

References

[1] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Beira, and A. Chechulin, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023.

[2] LangChain Inc., "LangGraph: Building stateful, multi-actor applications with LLMs," https://github.com/langchain-ai/langgraph, 2024, accessed: March 2025.

[3] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023.

[4] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.

[5] LLM-Agent Hallucination Survey Authors, "LLM-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions," *arXiv preprint arXiv:2509.18970*, 2025.

[6] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

[7] X. Liu, H. Yu, H. Zhang, Y. Xu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[8] G. Deng, Y. Liu, Y. Li, K. Wang *et al.*, "PentestGPT: An LLM-empowered automatic penetration testing framework," in *33rd USENIX Security Symposium*, 2024, pp. 6291–6308.

[9] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, and K. Ramchandran, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] Y. Bang, D. Chen, N. Lee, and P. Fung, "HalluLens: LLM hallucination benchmark," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[12] OWASP Foundation, "OWASP top 10 for agentic applications (2026)," https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/, 2026, accessed: April 2026.

[13] B. E. Strom, A. Applebaum, D. P. Miller *et al.*, "MITRE ATT&CK: Design and philosophy," https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf, 2022.

[14] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," National Institute of Standards and Technology, Tech. Rep. AI 100-1, 2023.

[15] ——, "NIST cybersecurity and AI profile (preliminary draft)," National Institute of Standards and Technology, Tech. Rep., 2025, preliminary draft, December 2025.

Mohammad Makchudul Alam is a Visiting Researcher with the DNS Research Lab, University College Dublin (UCD). His research interests include agentic AI reliability and failure characterisation in autonomous security operations.

Md. Redowan Zaman Anik is an Incident Handler with the Bangladesh Government e-GOV CIRT. His interests span SOC automation and the deployment of AI assistants in computer security incident response.

Sabrein Serag El Din el Bodour is a System Analyst with the Bangladesh Government e-GOV CIRT. His contributions to ARES focus on scenario grounding and failure-trace labelling.

Moshiul Islam is the Founder and CEO of EIC Limited, Dhaka, Bangladesh, a PCI-DSS Qualified Security Assessor (QSA) and Certified Forensic Examiner specialising in PCI-DSS audits and cybersecurity services across VAPT, SOC, and GRC. He contributed industry validation of ARES's agent-reliability requirements. Contact: moshiul@eicsecure.com.

Anca Delia Jurcut is an Assistant Professor with the School of Computer Science, University College Dublin. Her research interests include network security, anomaly detection, and trustworthy AI for cybersecurity.